Assignment 1 Report — CO1

Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

Name: Madhavan R

Register Number: 192421174

Repository: https://github.com/madhavanr13/DSA0109---C-

Scenario assumptions

Fares are Rs 10, Rs 20 and Rs 30 by farthest zone. Peak windows are half-open: 08:00–09:59 and 17:00–19:59. A journey is accepted only when the post-deduction balance is at least Rs 50. Money is represented as integer paise. The original PDF states students and senior citizens receive a concession but does not specify its percentage; Task 9's demonstration uses a clearly stated 50% illustrative concession, which must be replaced if the instructor specifies another rate.

Task 1

Analysis

Objects: SmartCard (private stored balance and validation/deduction), Passenger (identity and concession eligibility), Journey (zone/time), Zone (finite category), FareTable (single source of fares). Encapsulation protects balance; abstraction exposes canTravel/deduct; inheritance could specialize Student/Senior passenger; polymorphism could override concession policy. Rules: fare nonnegative, balance after deduction >= Rs 50, exact boundary peak windows [08:00,10:00) and [17:00,20:00), no deduction when denied. Procedural free functions and globals still run O(1) per trip and use O(1) balance storage, but changes are scattered and risk inconsistent rules. OOP centralizes invariants; code-change scope is smaller.

Source: Task01/main.cpp. See the task folder for code and actual execution output.

Task 2

Tokens and constants

Keywords: class, enum, constexpr, return. Identifiers: BASE_PAISE, Zone, fare. Constants: 1000, 20, 5000. Operators: =, *, /, ==. Punctuators: ; {} (). constexpr is typed, scoped and compile-time evaluable; #define is textual substitution without type/scope; const is immutable after initialization but not necessarily compile-time. enum class prevents implicit conversion to integers and avoids enumerator name leakage. Use UpperCamelCase types and UPPER_SNAKE_CASE constants. Macro vs constexpr has essentially no runtime or storage cost; constexpr provides safer diagnostics.

Source: Task02/main.cpp. See the task folder for code and actual execution output.

Task 3

Type choices

Use long long for aggregate paise and card balances; int for hour, zone index and percentage; bool for peak/concession; enum class Zone for zones; size_t for array indices; string for names. sizeof output is platform/compiler-specific; run the program to record local sizes. Money in integer paise avoids binary floating-point

representation error. <climits> supplies INT_MAX/LLONG_MAX. At 3,000,000 journeys × 3,600 paise, total is 10,800,000,000 paise, beyond a typical 32-bit int; long long is appropriate. double addition remains O(1), 8 bytes on this system, but repeated fractional additions can drift.

Source: Task03/main.cpp. See the task folder for code and actual execution output.

Task 4

Conversion analysis

Prediction before execution: 3.5 3.5? Correction by language rules: a/b performs integer division (3), then converts to double; explicit cast makes 3.5. char 'C' prints C; ch+1 promotes char to int and prints 68. bool initialized from 5 becomes true; default stream prints 1, boolalpha prints true. Comparing signed -1 to unsigned 1 converts -1 to unsigned, so comparison is false and output is s >= u. Full predicted output is recorded in prediction.txt before execution. Use static_cast for intended numeric conversion and brace initialization to reject narrowing. Explicit casts do not materially change runtime or storage; braces catch narrowing at compile time.

Source: Task04/main.cpp. See the task folder for code and actual execution output.

Task 5

Precedence prediction

Prediction before execution: 11 16; 9 6 4; 6 2; 3 1 -3 -1; 18. Multiplication precedes addition; parentheses override precedence. x++ yields old x=5 then increments; ++y increments 3 to 4 before use, so z=9, x=6, y=4. Conditional chooses max 6; % and * have same precedence and associate left-to-right: (6%4)*2=4. Integer division truncates toward zero, and remainder sign follows dividend: -10/3=-3, -10%3=-1. Loop adds 0+3+6+9=18. Avoid i=i++ because it is error-prone and historically undefined/unsequenced depending on language context. Parenthesize peak ranges for readability.

Source: Task05/main.cpp. See the task folder for code and actual execution output.

Task 6

Pseudocode

BEGIN

READ zone, hour, balance

SELECT zone: Inner→1000; Middle→2000; Outer→3000; otherwise reject

peak ← (hour >= 8 AND hour < 10) OR (hour >= 17 AND hour < 20)

IF peak THEN fare ← fare × 120 / 100

IF balance – fare >= 5000 THEN deduct fare and print fare/balance ELSE deny

END

Switch is clear for finite zones; if-else suits range checks; ?: suits short value selection. A counted for loop suits a known trip array; range-for is safest for arrays/containers; while suits input retry. break exits a loop/switch; continue skips invalid trips. No matching switch case means no action unless default handles invalid values. Both switch and array lookup are O(1); array uses space for zone entries and scales by adding an entry, while switch needs another case.

Source: Task06/main.cpp. See the task folder for code and actual execution output.

Task 7

Comparison

Procedural functions accept a balance reference and fare; callers can still alter the balance directly. SmartCard owns private balance and permits only validated deductions. Both are O(1) time per operation and O(1) memory per card. Private state prevents arbitrary code from setting balance negative or bypassing the Rs 50 minimum. A public balance changes neither asymptotic complexity nor memory, but breaks the invariant. Centralizing MIN_BALANCE makes policy changes easier.

Source: Task07/main.cpp. See the task folder for code and actual execution output.

Task 8

Safe I/O

cin >> is convenient for typed input; if extraction fails, test cin.fail(), call cin.clear(), then ignore through newline using numeric_limits<streamsize>::max(). getline plus stoi processes a line in O(L) time and O(L) buffer space, allows full-line validation and catches invalid_argument/out_of_range; formatted extraction also scans input but token-based recovery needs clearing and discarding. setw sets the next field width; setfill sets padding; fixed/setprecision format decimal output. Always validate hour 0–23 and zone before computing a fare.

Source: Task08/main.cpp. See the task folder for code and actual execution output.

Task 9

Test evidence

The test harness compares actual paise against expected paise. Cases include all zones, peak/off-peak and boundaries 08:00, 10:00, 17:00, 20:00, plus concession. Unknown zones, negative recharge and nonnumeric hours should be rejected by the input-validation layer; do not cast an unchecked zone index into an enum. A table-driven loop processes t tests in O(t) time and O(t) storage for a table; adding a boundary case is one row. Five sample journeys can be hand-totalled as 1000+2400+3000+3600+3000 = 13000 paise (Rs 130.00), before applying card eligibility.

Source: Task09/main.cpp. See the task folder for code and actual execution output.

Task 10

Measurements and comparison

Run with g++ -std=c++17 -Wall -Wextra -Wconversion. At 3,000,000 journeys × Rs 36, total = Rs 108,000,000 (10,800,000,000 paise). This exceeds INT_MAX on a common 32-bit int, so signed int overflow is not safe (signed overflow is undefined behavior). long long safely holds this total; double can represent this integer-sized total with ample precision for ordinary use, but decimal fractional additions can introduce rounding. Per-addition time is O(1); int usually 4 bytes, long long and double commonly 8 bytes. Record actual warnings from the compiler used; warnings can differ by compiler/version.

Source: Task10/main.cpp. See the task folder for code and actual execution output.

Task 11

Recommendation

Use enum class Zone, constexpr integer-paise fares, long long balances/totals, a private SmartCard balance and centralized fare/peak functions. Define peak intervals explicitly as half-open ranges. Fare revision changes one table/constant and requires regression tests. Limitations: compile-time fare updates require rebuilding; concession rules and monthly passes need explicit policy; persistence, concurrency, audit logging and transaction rollback are outside this small model. O(1) work per journey remains appropriate for millions of journeys. A configuration file adds O(z) startup reading and O(z) memory but avoids recompilation for fare-only revisions; validate file contents before opening gates.

Source: Task11/main.cpp. See the task folder for code and actual execution output.

Task 12

Design challenge

Impact: Zone gains Airport and Count; fare table grows with Airport=6000 paise; switch cases or index table change; fare policy separates weekday peak surcharge from weekend discount; MonthlyPass stores two covered zones. Retain SmartCard's encapsulation but modify fare calculation and add a Pass class; replace switch with indexed std::array to reduce repeated branches and make adding a zone a table entry plus enum value. Policy chosen: weekend discount takes precedence and does not combine with peak surcharge. Worked examples: Airport weekday off-peak Rs 60; weekday peak Rs 72; weekend even during peak hours Rs 54. A two-zone pass for Inner and Airport covers those zones but not Middle. Tests show table-driven lookup is O(1), uses O(z) fare storage, and the class protects balance invariants. For production, validate enum indices, handle pass expiry/renewal, and define whether a pass overrides stored-value deductions.

Source: Task12/main.cpp. See the task folder for code and actual execution output.

Build and validation procedure

Compile with g++ -std=c++17 -Wall -Wextra -Wconversion main.cpp -o main. Each task program is compiled and executed in the local environment. Actual outputs are saved as output.txt; a terminal-style PNG is saved as output.png. Task 4 and Task 5 prediction files were authored before running those programs.

Conclusion

The safest baseline design uses integer paise, wide integer totals, strongly typed zones, named compile-time constants, explicit half-open peak-hour checks, and a SmartCard class with private balance. The extended design uses a zone-indexed fare table and separates weekend discount policy from weekday peak surcharge. The monthly pass is represented by a separate class, keeping fare calculation and card-balance invariants independently testable.

Student verification checklist

- ■ Replace name/register number and repository URL placeholders.
- ■ Read and understand each source file; adapt and rerun it yourself.
- ■ Confirm every test and output on your own machine.
- ■ Take genuine screenshots of your terminal/IDE execution if your faculty requires screenshots of your own run.
- ■ Create/push the GitHub remote, add faculty access if private, and submit link/report in Google Classroom.